

Chapter 6

1) Explain Triples with example.

Triples in Intermediate Code

Triples is a representation of intermediate code used in compilers.

It avoids using temporary variable names and instead refers to results by their **position (index)** in the list.

Structure of Triples

Each triple has 3 fields:

1. Operator (op)
2. Argument 1 (arg1)
3. Argument 2 (arg2)

👉 Results are referred using the **index number** of the triple.

Example

Expression:

```
a = b + c * d
```



Step 1: Convert to Intermediate Form

```
t1 = c * d
t2 = b + t1
a = t2
```



Step 2: Triples Representation

Index	Operator	Arg1	Arg2
0	*	c	d
1	+	b	(0)
2	=	a	(1)

👉 (0) means result of index 0

👉 (1) means result of index 1

2) Explain any five code optimization techniques in compiler designing with suitable example.

Code Optimization Techniques in Compiler Design

Code optimization improves program efficiency by reducing execution time and memory usage.

1. Constant Folding

👉 Compile-time evaluation of constant expressions.

Example:

```
x = 5 + 10;
```

Optimized:

```
x = 15;
```

2. Constant Propagation

👉 Replaces variables with their constant values.

Example:

```
a = 10;
```

```
b = a + 5;
```

Optimized:

```
b = 10 + 5;
```

3. Dead Code Elimination

👉 Removes code that does not affect program output.

Example:

```
x = 10;
```

```
y = 20;
```

```
return x;
```

Optimized:

```
x = 10;
```

```
return x;
```

4. Common Subexpression Elimination

👉 Avoids recomputing same expressions.

Example:

```
t1 = a + b;
```

```
t2 = a + b;
```

Optimized:

```
t1 = a + b;
```

```
t2 = t1;
```

5. Loop Optimization (Loop Invariant Code Motion)

👉 Moves constant computations outside loop.

Example:

```
for(i = 0; i < n; i++) {  
    x = a + b;  
}
```

Optimized:

```
x = a + b;  
for (i = 0; i < n; i++) {  
}
```

5. Strength Reduction-

In this technique,

- As the name suggests, it involves reducing the strength of expressions.
- This technique replaces the expensive and costly operators with the simple and cheaper ones.

Example-

Code Before Optimization	Code After Optimization
$B = A \times 2$	$B = A + A$

1. Constant Folding CF

2. Constant Propagation CP

3. Dead Code Elimination DCE

4. Common Subexpression Elimination CSE

5. Loop Optimization (Loop Invariant Code Motion) LO

6. Strength Reduction SR

3) Describe the different ways of representing Intermediate Code? Explain with suitable example.

Intermediate codes can be represented in a variety of ways

The intermediate code can be represented in the form of postfix notation, syntax tree, three address codes, Quadruples, and triples.

Postfix Notation

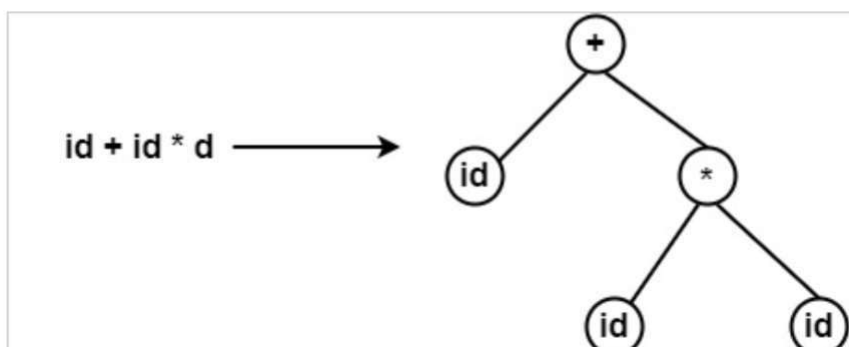
In postfix notation, the operator comes after an operand, i.e., the operator follows an operand.

Example

Postfix Notation for the expression $(a+b) * (c+d)$ is $ab + cd + *$

Syntax Trees

It is condensed form of parse tree in which leaves are identifiers and interior node will be operators.



Three Address Code–

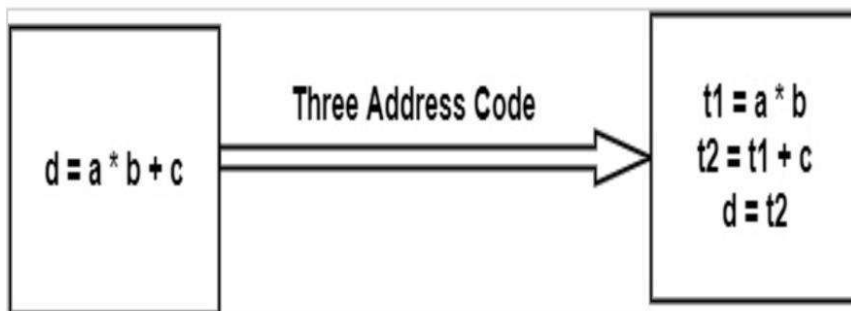
These are statements of form $c = a \text{ op } b$, i.e., in which there will be at most three addresses, i.e., two for operands & one for Result.

Each statement has the general form of:

Syntax:

$$\text{result} := \text{operand}_1 \text{ operator operand}_2$$

Example of three address code for the statement



Three-address code can be represented in two forms:

quadruples and triples.

1. Quadruples

👉 Each instruction has **4 fields**:

- Operator (op)
- Argument1 (arg1)
- Argument2 (arg2)
- Result

Example

Index op arg1 arg2 result

0 * b c t1

1 + a t1 t2

2 = t2 — x

👉 Uses **temporary variables (t1, t2)**

2. Triples

👉 Each instruction has **3 fields**:

- Operator
- Argument1
- Argument2

👉 Result is referred by **index number**

Example

Index op arg1 arg2

0 * b c

1 + a (0)

2 = x (1)

👉 (0) → result of index 0

👉 (1) → result of index 1

Key Difference

- Quadruples → Uses temporary variables
- Triples → Uses index references (no temporaries)

4) Explain different issues in the code generation phase of Compiler.

Issues in Code Generation

The main goal of a code generator is to produce **correct and efficient machine code**.

The following issues arise:

1. Input to Code Generator

👉 The code generator takes input from the **intermediate code (IC)** and **symbol table**.

- Intermediate code tells *what operations to perform*
- Symbol table gives *information about variables (type, size, location)*

Example:

t1 = a + b

👉 Code generator must know:

- a, b → variables
 - t1 → temporary value
-

2. Target Program

👉 The output should be suitable for the **target machine (CPU)**.

- Can be **assembly code** or **machine code**
- Must follow machine's instruction set

Example:

a = b + c

👉 For one machine:

LOAD R1, b

ADD R1, c

STORE a

👉 Another machine may use different instructions

3. Memory Management

👉 Deciding where variables and data will be stored.

- Stack (local variables)
- Heap (dynamic memory)
- Registers (fast access)

Example:

```
int a = 10;
```

👉 Compiler decides:

- Store a in register or memory location
-

4. Instruction Selection

👉 Choosing the best machine instructions.

- Many ways to perform same task
- Choose fastest/shortest one

Example:

```
ADD R1, 1
```

can be replaced by:

```
INC R1 (faster)
```

5. Register Allocation

👉 Deciding which variables go into registers.

- Registers are limited
- Important variables should be kept in registers

Example:

```
x = a + b;
```

```
y = x + c;
```

👉 Keep x in register to avoid memory access again

6. Evaluation Order

👉 Deciding order of evaluating expressions.

- Affects register usage and performance

Example:

$(a + b) + (c + d)$

👉 Better evaluation:

- Compute $(a + b)$ first
- Then $(c + d)$
- Then add results

👉 This reduces register usage and improves efficiency

Short Conclusion (for exam)

Code generation faces issues like handling input, selecting instructions, managing memory, allocating registers, and deciding evaluation order to produce efficient machine code.

1. Input to Code Generator

2. Target Program

3. Memory Management

4. Instruction Selection

5. Register Allocation

6. Evaluation Order

Peephole

It is machine dependant

<https://www.geeksforgeeks.org/compiler-design/peephole-optimization-in-compiler-design/>

Lec 24: Peephole Optimization in Compiler | Dead code elimination | Strength reduction

1) Redundant load and store

```
a = b + c
d = a + e
```

Assume a, b, c

```
Mov b, R0
Add C, R0
Mov R0, a
Mov a, R0
Add e, R0
Mov R0, d
```

2) Strength Reduction

```
x * x ⇒ x * x
x * 2 ⇒ left shift
x * 2 ⇒ right shift
```

3) Simplify Algebraic Expressions

```
a = a + 0
a = a * 1
a = a / 1
a = a - 0
```

4) Replace slower instructions with faster

```
x Add #1, R ⇒ INC R ✓
Sub #1, R ⇒ DEC R ✓
```

5) Deadcode Elimination

```
int dead(void)
{
    int a = 10;
    int b = 20;
    int C;
    C = a * 10;
    return C;
    b = 30;
    b = b * 10;
    return 0;
}
```

→ return C; (dead code)

Basic blocks

<https://www.geeksforgeeks.org/compiler-design/basic-blocks-in-compiler-design/>